

Where *DetailedBalanceChecker* Fits in the Literature

A Competitive Analysis: Methods, Tools, and Honest Appraisal

May 2026

Abstract

This document provides a critical survey of every approach that competes with, complements, or substitutes for *DetailedBalanceChecker* — a tool for the symbolic, algebraic verification of the detailed-balance condition in Monte Carlo algorithms. The analysis covers statistical testing methods, probabilistic programming language tools, model checkers, formal verification systems, physics-domain validation tools, and software libraries. The conclusion is that *DetailedBalanceChecker* is genuinely unique in its specific niche: no existing tool provides automated symbolic DB verification of physics MC algorithms with free coupling constants. However, the niche is narrow, the Mathematica dependency is a serious adoption barrier, and the community being targeted has almost no culture of formal algorithm verification. The analysis identifies where *DetailedBalanceChecker* would be rejected by reviewers, where it would struggle to be used in practice, and what it would need to achieve broad impact.

Contents

1	What we are comparing against	3
2	Category 1: Statistical testing of MCMC correctness	3
2.1	Geweke (2004) joint-distribution test	3
2.2	Simulation-Based Calibration (SBC)	3
2.3	MCUnit / Gandy & Scott (2020)	3
2.4	Wallerberger & Gull (2017): physics-domain testing	4
2.5	Tjelmeland & Kvaløy (2025): catching bugs in published samplers	4
2.6	Physical_validation (Shirts et al.)	4
3	Category 2: Probabilistic programming with correctness guarantees	4
3.1	Hakaru (Narayanan et al. 2016)	4
3.2	Gen.jl (Cusumano-Towner et al.)	5
3.3	PSI Solver (Gehr et al., ETH Zurich)	5
4	Category 3: Model checking	5
4.1	PRISM and Storm	5
5	Category 4: Formal verification (theorem provers)	6
5.1	Isabelle/HOL and Lean/Mathlib	6
6	Category 5: Involutive MCMC and structural guarantees	6
7	Category 6: Balance Condition Library and related software	7
7.1	BCL — Balance Condition Library (CMSI, University of Tokyo)	7
7.2	MCMCDebugging.jl	7
8	The structural gap: no symbolic DB checker exists for physics MC	7

9	Critical assessment: where <i>DetailedBalanceChecker</i> would struggle	7
9.1	The Mathematica dependency is a serious barrier	7
9.2	The state space limitation is a real constraint, not just a caveat	8
9.3	The target community has almost no verification culture	8
9.4	The rightward-cluster example may be the only truly compelling case	9
9.5	The Mathematica trust model creates a meta-level question	9
9.6	JCTC reviewers will compare against involutive MCMC	9
9.7	The BFS cost limits the complexity of testable algorithms	9
9.8	LLM loop section is prospective, and this weakens the paper	10
10	Where <i>DetailedBalanceChecker</i> would receive genuine adoption	10
11	What would make the paper significantly stronger	10
12	Summary verdict	11

1 What we are comparing against

DetailedBalanceChecker claims to verify the detailed-balance (DB) condition $T(i \rightarrow j) \pi(i) = T(j \rightarrow i) \pi(j)$ for Monte Carlo algorithms by: (1) intercepting all random calls and reducing them to reads from a binary tape, (2) enumerating all execution paths by BFS to produce a symbolic transition matrix, (3) checking the DB condition algebraically with coupling constants as free symbols, giving a proof valid for all parameter values simultaneously. It is implemented in Mathematica, targets physics simulation algorithms (Kawasaki, VMMC, cluster moves) on lattices of up to roughly ten sites per dimension, and is the only tool of its kind.

The honest question is: *does anyone need this, and is there a better way to get the same thing?*

2 Category 1: Statistical testing of MCMC correctness

This is by far the most developed category, and its tools are what practitioners actually use.

2.1 Geweke (2004) joint-distribution test

The most widely cited MCMC correctness test [1]. If a Bayesian sampler is correct, forward simulation (draw θ from prior, x from likelihood) and MCMC alternation must produce matching marginals on every test statistic; a χ^2 or KS test detects divergence. Correct machinery, clearly described, well-understood properties.

Assessment relative to *DetailedBalanceChecker*: The Geweke test is statistical, not formal. It cannot certify correctness; it can only fail to detect a violation. It also requires a Bayesian structure (known prior and likelihood), which physics simulation algorithms do not have. For the specific use case of verifying VMMC or Kawasaki dynamics with free coupling constants, Geweke is completely inapplicable: there is no “prior” over coupling constants, and the stationary distribution is known only up to a partition function that depends on all couplings.

But here is the real problem: In the statistics and machine learning community, which is far larger than the physics simulation community, Geweke-style tests are entirely sufficient for almost all applications. Most Bayesian samplers (Stan, PyMC, Turing.jl) operate on well-specified models where the joint distribution is known. *DetailedBalanceChecker* has nothing to offer this much larger community.

2.2 Simulation-Based Calibration (SBC)

Talts et al. [2] showed that if parameters θ^* are drawn from the prior and data x^* from the likelihood, the rank of θ^* among posterior samples must be uniform. Implemented in Stan’s workflow and the R package *SBC*. Well-adopted in Bayesian statistics. Strictly more informative than Geweke for detecting subtle biases.

Assessment: Same fundamental barriers as Geweke: Bayesian structure required, statistical not formal. Not applicable to physics MC.

2.3 MCUnit / Gandy & Scott (2020)

Gandy & Scott [3] introduced the *Exact Invariance Test*: a sequential hypothesis test for whether a given Markov chain has a specified invariant distribution, with guaranteed exact p -values (i.e., the false rejection probability is controlled non-asymptotically). Implemented as the R package *mcunit* and Julia package *MCMCDebugging.jl*.

Assessment: More rigorous than Geweke in the sense that p -values are exact, but still fundamentally statistical. Tests the *invariant distribution*, not DB directly. A sampler that breaks DB but preserves the invariant distribution (as in the rightward-cluster example) would pass. Requires an explicitly computable target distribution. Not applicable to physics MC algorithms.

2.4 Wallerberger & Gull (2017): physics-domain testing

The most relevant competitor from the physics domain [4]. Uses exact analytical identities (Schwinger–Dyson equations, Ward identities) satisfied by the Ising model and the Anderson impurity model to test whether a simulation produces statistically consistent output. Code released on GitHub (CQMP/HypothesisTesting). Applicable to large production simulations.

Assessment: Tests algorithm *outputs* against known analytical benchmarks — cannot test *novel* algorithms for which no identities are known. Statistical, not formal. The approach becomes vacuous precisely when the algorithm is new and when the absence of known identities is the problem. *DetailedBalanceChecker* can test novel algorithms without any analytical benchmark, which is the case that actually matters for new algorithm development.

However: For practitioners working with the Ising model or known integrable systems, Wallerberger–Gull is far more convenient, scalable to large systems, and requires no algorithm re-implementation. In this regime *DetailedBalanceChecker* is both less convenient and less informative.

2.5 Tjelmeland & Kvaløy (2025): catching bugs in published samplers

Recent work [5] found a bug in a G-Wishart sampler that had been in the literature for years. The method: initialize an MH chain with a sample from the claimed sampler, run fixed steps, exploit exchangeability at stationarity. This is a practical demonstration that statistical tests can find real bugs in published algorithms.

Assessment: An important reminder that statistical tests *do* catch real bugs, and that the literature contains wrong algorithms. The question is whether *DetailedBalanceChecker*’s formal approach catches bugs that statistics misses. The rightward-cluster example (Section ??) provides one clear case where statistical tests fail categorically. Whether this class of bugs is common in practice is unknown.

2.6 Physical_validation (Shirts et al.)

The `physical_validation` Python package [6] validates molecular dynamics simulations by checking energy conservation, kinetic energy distributions, and ensemble consistency across multiple state points. Integrated into GROMACS’s automated testing pipeline.

Assessment: Targets MD, not MC. Does not check DB. But represents the kind of practical, automated testing infrastructure that production simulation software actually adopts. The contrast with *DetailedBalanceChecker*’s Mathematica-based workflow is stark: `physical_validation` is a `pip install` that integrates with existing Python workflows and runs on production trajectories from GROMACS. This is the adoption model that works.

Critical point: The physics simulation community has shown it will adopt automated testing tools when they integrate into existing workflows and operate on production-scale data. A Mathematica tool that requires algorithm re-implementation and only works for $\lesssim 10$ sites does not fit this model.

3 Category 2: Probabilistic programming with correctness guarantees

3.1 Hakaru (Narayanan et al. 2016)

Hakaru [7] is a Haskell-embedded PPL that generates Metropolis–Hastings kernels automatically via a *disintegration* transformation. The guarantee: any MH kernel produced by Hakaru satisfies DB *by construction*, assuming the transformation is sound. This is a correct-by-construction approach rather than a post-hoc checker.

Assessment: In the MH framework, Hakaru is strictly superior to *DetailedBalanceChecker*: it provides a proof by construction, without any BFS enumeration, for any model expressible in

Hakaru’s language. It does not check externally written algorithms, cannot verify VMMC or cluster moves, and has not been applied to physics simulation. The key insight from `reversible_checker.tex` applies here: Hakaru constructs MH kernels as involutions on an extended state space, which structurally guarantees DB [11]. *DetailedBalanceChecker* checks whether an externally written algorithm happens to satisfy DB; Hakaru builds one that must.

But: Hakaru appears to be a research prototype, not production software. The GitHub repository (`hakaru-dev/hakaru`) was last actively updated in 2019. The disintegration transformation is only proven sound for a specific fragment of the language. Application to complex physics algorithms (which have non-standard link probabilities, frustration conditions, and Gaussian proposals) would require significant new language development.

3.2 Gen.jl (Cusumano-Towner et al.)

Gen [8] is a Julia PPL supporting custom MCMC kernels. It provides runtime checks (`check=true`) that detect “common bugs that break stationarity.” The involutive MCMC extension [9] provides structural DB guarantees for kernels expressed as involutions. In active production use.

Assessment: Gen’s involutive MCMC is elegant and correct-by-construction for algorithms expressible in that framework. Runtime checks are better than nothing. But: (1) only applies to algorithms written within Gen; (2) runtime checks are statistical, not formal; (3) VMMC’s cluster-building logic with frustrated links is not naturally an involution on a simple extended state space.

3.3 PSI Solver (Gehr et al., ETH Zurich)

PSI [10] computes exact symbolic posterior distributions from probabilistic programs using symbolic integration. It produces rational function representations of posteriors and outperforms Mathematica/Maple on certain simplifications. Actively maintained, open source.

Assessment: PSI is the closest tool to *DetailedBalanceChecker* in the symbolic/algebraic dimension. In principle, one could write a transition probability computation $T(i \rightarrow j)$ as a PSI program and have PSI produce the symbolic expression. In practice: PSI is designed for Bayesian inference in models, not for analyzing imperative MC algorithm transition structures. It has no mechanism for systematically enumerating all execution paths of a random algorithm and constructing a transition matrix. The gap between “symbolic posterior inference for a model” and “symbolic transition matrix for an algorithm” is substantial.

Critical point: If someone with PSI expertise wanted to build a DB checker, PSI would be a better foundation than Mathematica in several respects: open source, actively maintained, proven inference machinery. The Mathematica implementation of *DetailedBalanceChecker* uses `Block/UpValue` in a way that is genuinely hard to replicate outside Mathematica, which is both an advantage (robust interception) and a long-term risk (proprietary dependency, maintenance).

4 Category 3: Model checking

4.1 PRISM and Storm

PRISM [14] and Storm [15] are mature, actively maintained probabilistic model checkers. Storm’s parametric analysis produces rational functions of symbolic parameters for reachability and steady-state properties of Markov chains. Both are open source and have Python APIs.

Assessment: In principle, if one manually encoded an MC algorithm as a parametric PRISM/Storm model, these tools could compute symbolic transition probabilities, from which DB checking is a post-processing step. In practice, no published work does this for any physics MC algorithm. The barriers are:

1. Manual model construction: translating a VMMC algorithm from Mathematica (or C++) to the PRISM/JANI input language is a significant manual effort.

2. Property specification: DB is an algebraic equality of transition products, not a PCTL/CSL formula. Verification requires a post-processing step outside the model checker.
3. No random-call interception: Storm does not intercept `RandomReal []` or analogous calls and build an execution tree automatically. That is the core innovation of *DetailedBalanceChecker*.

Verdict: Storm+parametric analysis is the closest technically capable alternative, but the gap between “capable in principle” and “usable in practice” is large. A Storm-based DB checker would need all of *DetailedBalanceChecker*’s innovations applied to a Storm front-end. It would be a better long-term foundation (open source, larger team, more users) but does not currently exist.

5 Category 4: Formal verification (theorem provers)

5.1 Isabelle/HOL and Lean/Mathlib

Hölzl’s Isabelle/HOL formalization of Markov chains [16] and the Lean/Mathlib probability library (2024, arXiv:2510.04070) provide the foundational infrastructure for machine-checked proofs about Markov chains. In principle, one could prove DB for a specific MCMC algorithm in Isabelle or Lean with full machine checking.

Assessment: A machine-checked Isabelle proof would be strictly stronger than *DetailedBalanceChecker*’s CAS-algebra output: it would provide an absolute formal guarantee. In practice: no published Isabelle, Coq, or Lean proof of DB exists for any nontrivial MCMC algorithm. The Lean Mathlib work formalizes Markov kernels and basic reversibility theory but has not produced proofs for specific algorithms. The effort required to prove DB for VMMC in Lean is a multi-year research project in its own right.

Critical point: *DetailedBalanceChecker* occupies a middle ground between “statistical test” (fast, scalable, no proof) and “formal theorem prover” (complete proof, human-years of effort). The CAS approach gives a proof at low cost. This middle-ground position is exactly right for the use case, but it is worth being honest: CAS algebra is not machine-checked in the Lean/Coq sense. If Mathematica’s `FullSimplify` returns zero due to an algebraic identity, that is a proof; if it returns zero due to a CAS bug, it is not. The trust model of *DetailedBalanceChecker* is fundamentally different from formal theorem provers.

6 Category 5: Involutive MCMC and structural guarantees

The most important insight from `reversible_checker.tex` is the hierarchy:

$$\underbrace{\text{bijection}}_{\text{reversible language}} \subsetneq \underbrace{\text{involution} + \text{uniform aux}}_{\text{DB w.r.t. uniform}} \subsetneq \underbrace{\text{correct acceptance w.r.t. } \pi}_{\text{DB for target } \pi}.$$

Neklyudov et al.’s involutive MCMC framework [11] shows that any iMCMC chain (algorithm expressible as a measure-preserving involution on an extended state space) automatically satisfies DB. Tierney [12] and Duane et al. [13] showed that MH and HMC are both instances. Gen’s involutive MCMC extension provides a programming interface for specifying such algorithms in Julia.

Assessment: The involutive framework provides a sufficient structural condition for DB that is strictly stronger than what *DetailedBalanceChecker* offers: if you can prove your algorithm is an involution, you have DB without any BFS. However:

1. Many algorithms of interest (particularly VMMC with its frustrated-link cluster building) are *not* naturally expressible as simple involutions on a finite extended state space.
2. The involution condition gives DB with respect to the *Lebesgue measure* on the extended space; obtaining DB with respect to a specific π requires additionally that the acceptance ratio is correctly computed.

3. Even if VMMC can be expressed as an involution, **verifying that a specific implementation correctly implements the involution requires a separate check** — which is exactly what *DetailedBalanceChecker* provides.

Bottom line: The involutive framework and *DetailedBalanceChecker* are complementary, not competing. The involutive framework provides a design principle; *DetailedBalanceChecker* verifies implementations. For algorithms not naturally expressible as involutions (which includes VMMC), the involutive framework offers no direct path to correctness.

7 Category 6: Balance Condition Library and related software

7.1 BCL — Balance Condition Library (CMSI, University of Tokyo)

The MateriApps software catalog lists a “Balance Condition Library” (BCL) from the Computational Materials Science Initiative at the University of Tokyo. The description is minimal and documentation sparse; it appears to be a utility for computing and verifying balance conditions in quantum Monte Carlo contexts, possibly related to the ALF/QUEST QMC packages.

Assessment: This is the closest named software to *DetailedBalanceChecker* in the physics simulation domain. However, based on available documentation, BCL appears to check balance conditions numerically at specific parameter values, not symbolically. If this is correct, it is fundamentally weaker than *DetailedBalanceChecker*. **This requires further investigation before claiming priority.**

7.2 MCMCDebugging.jl

A Julia package in the TuringLang ecosystem that implements the Gandy–Scott exact invariance test. Not a symbolic verifier; extends the statistical testing approach to Julia. Open source, actively maintained.

Assessment: Inferior to *DetailedBalanceChecker* for formal DB verification. Superior for practitioners who want a quick, scalable statistical test within Julia workflows.

8 The structural gap: no symbolic DB checker exists for physics MC

After exhaustive search, *no existing tool provides what DetailedBalanceChecker provides*: automated extraction of a symbolic transition matrix from an existing MC algorithm, algebraic verification of DB with free coupling constants, for physics simulation algorithms (cluster moves, Kawasaki dynamics, VMMC) without a Bayesian model structure.

Table 1 summarises the landscape.

9 Critical assessment: where *DetailedBalanceChecker* would struggle

Despite being unique, *DetailedBalanceChecker* faces substantive problems that reviewers, users, and competitors will raise. These are listed without softening.

9.1 The Mathematica dependency is a serious barrier

Mathematica costs \$400/year for individuals and \$2500+/year for institutions. The physics simulation community predominantly uses C++, Python, and Julia, and large fractions of computational physics groups at universities do not have institutional Mathematica licenses. **A tool that requires Mathematica cannot achieve broad adoption in a community that does not use Mathematica.**

Table 1: Comparison of approaches to MCMC correctness verification. FP = formal proof. Phys = applicable to physics MC. Free params = works for all parameter values simultaneously. Auto = no manual model construction required.

Method	FP?	Phys?	Free params?	Auto?	Status
Geweke (2004)	No	No	No	Yes	Widely used
SBC / Cook-Gelman	No	No	No	Yes	Widely used
MCUnit / Gandy-Scott	No	No	No	Yes	Active (R/Julia)
Wallerberger-Gull (2017)	No	Partial	No	Yes	Code on GitHub
Tjelmeland-Kvaløy (2025)	No	No	No	Yes	Very recent
Hakaru (2016)	Yes (constr.)	No	Limited	Partial	Appears dormant
Gen.jl iMCMC	Partial	No	No	Partial	Active
PSI Solver	Yes (symbolic)	No	Yes	Partial	Active
PRISM / Storm	Yes (parametric)	Theoretically	Yes	No	Active
Isabelle/HOL	Yes (full)	Theoretically	Yes	No	No proofs yet
physical.validation	No	MD only	No	Yes	Active, GROMACS
BCL (CMSI)	Unclear	Yes (QMC)	Unknown	Unknown	Sparse docs
DBC (this work)	Yes (CAS)	Yes	Yes	Yes	Active

The Block/UpValue mechanism is genuinely Mathematica-specific and hard to replicate, which is why it was chosen. But from an adoption perspective, this is an anchor. PSI Solver, Storm, and Gen.jl are all open-source and pip/Julia-installable. *DetailedBalanceChecker* is not.

The paper’s discussion section acknowledges Julia/Cassette.jl as an alternative, but that port does not exist yet. Reviewers will ask why the paper introduces a tool in a proprietary language when open-source alternatives are feasible.

9.2 The state space limitation is a real constraint, not just a caveat

The claim “detailed-balance violations are a property of the algorithm, not the system size” is *mostly* true but not always. Correctness on a 2×2 lattice does not automatically imply correctness on a 200×200 lattice for all algorithm variants. Algorithms that depend on the system size through modular arithmetic, boundary conditions, or indexing can have bugs that only appear above a threshold size. The missing-direction VMMC example already demonstrates this: the algorithm passes on 2×2 but fails on 3×3 precisely because of boundary condition effects. **This undercuts the “small system suffices” argument.**

The correct claim is more nuanced: structural bugs (wrong acceptance rule, missing reverse move) manifest on small systems, but implementation bugs (wrong modular arithmetic, off-by-one in boundary conditions) may not. This distinction should be made explicit and honest in any publication.

9.3 The target community has almost no verification culture

In the physics simulation community, the dominant practice for algorithm verification is: (1) derive the algorithm analytically, (2) test on known systems at known parameters. The idea of formal algorithmic verification is largely unknown. The schism-pl/vmmc Rust repository (a production VMMC implementation) is self-described as “probably correct.” libVMMC has no automated DB check. Most VMMC users do not know that the missing-direction variant would pass numerical tests.

There is a risk that no one uses the tool because no one knows they need it. This is not a criticism of the tool’s correctness, but of its adoption path. A tool that finds real bugs in widely used production

code would rapidly build adoption. A tool that proves that correct algorithms are correct does not.

9.4 The rightward-cluster example may be the only truly compelling case

The paper’s central diagnostic example — an algorithm that passes all numerical tests while violating DB — is the rightward-only cluster algorithm. This is a compelling illustration, but it may be a constructed example rather than a real bug found in a production algorithm. Reviewers at JCTC will ask: *has this tool found a bug in a real, published algorithm?*

If the answer is no, the paper’s impact is significantly reduced. The strongest possible submission would include a case where *DetailedBalanceChecker* finds a DB violation in a published or widely distributed algorithm that numerical tests would have missed. The G-Wishart case (Tjelmeland–Kvaløy 2025) showed this is possible with statistical methods; the symbolic approach should be able to do the same.

9.5 The Mathematica trust model creates a meta-level question

The claim is that a PASS from *DetailedBalanceChecker* constitutes a “formal proof.” This claim rests on: (1) Mathematica’s FullSimplify correctly proving algebraic identities; (2) the BFS correctly enumerating all paths; (3) the random-call interception correctly capturing all randomness. Each of these is trusted but not verified.

This is not a theoretical objection — CAS systems are extensively tested and rarely wrong on algebraic identities. But the claim of “formal proof” should be made carefully. A Lean or Coq proof is machine-checked by a kernel small enough to be trusted; Mathematica’s FullSimplify is a large, complex system with bugs (Wolfram’s bug tracker is public and has entries for FullSimplify incorrectly proving identities on edge cases).

The more honest language is “algebraic proof via CAS” rather than “formal proof.” This is a stronger statement than “statistical test” but weaker than “machine-checked proof.” The paper should be explicit about this.

9.6 JCTC reviewers will compare against involutive MCMC

The involutive MCMC framework (Neklyudov et al. 2020) shows that any algorithm expressible as an involution on an extended state space automatically satisfies DB. A referee with this background will ask: “why not just implement VMMC as an involution and get DB for free?”

The answer is that: (a) VMMC’s cluster-building logic with frustrated links is not trivially an involution; (b) even if it were, verifying that a specific implementation correctly implements the involution requires a separate check; and (c) the involutive framework gives DB with respect to the uniform measure on the extended space, not necessarily with respect to a specific π if the acceptance ratio is wrong.

But the paper must engage seriously with Neklyudov et al. [11], which is not currently cited.

9.7 The BFS cost limits the complexity of testable algorithms

A VMMC algorithm on a 3×3 lattice with abstract coupling fields takes 12.5 minutes on 4 kernels. On a 4×4 lattice this would be estimated $\sim 1000\times$ longer — months on 4 kernels. **The exponential scaling wall is fundamental, not implementational.**

For real VMMC simulations, users work on 50×50 or larger lattices. The argument “small system suffices to prove structural correctness” is sound but must be presented carefully, since the bugs that users actually introduce in production code are often precisely the boundary-condition and indexing bugs that require larger systems to manifest.

9.8 LLM loop section is prospective, and this weakens the paper

The use-case section describing LLM-assisted algorithm discovery is entirely prospective. JCTC reviewers will likely view this as speculative and will suggest it be removed or substantially shortened. The paper is stronger if focused on what the tool demonstrably does: verify existing algorithms. The LLM loop section risks making the paper look like it is overselling.

10 Where *DetailedBalanceChecker* would receive genuine adoption

Despite the above criticisms, there are scenarios where *DetailedBalanceChecker* would be genuinely valued.

- 1. As a development tool for new cluster algorithms.** Researchers proposing new cluster algorithms (variants of VMMC, new rejection-free schemes, algorithms for non-standard geometries) currently have to prove DB analytically, which is tedious and error-prone. *DetailedBalanceChecker* automates this. Any group developing a new algorithm and needing to certify it would benefit from the tool. This is a small but real community.
- 2. As a sanity check for algorithm implementations.** Even when the algorithm is known to be correct analytically, its implementation may contain subtle bugs. The missing-direction VMMC example shows that a single-line change in the displacement list can break DB in a way that no statistical test detects. Groups that release simulation software (e.g., libVMMC, HOOMD’s patch-particle MC) could run *DetailedBalanceChecker* on their implementations as a test.
- 3. For teaching and graduate training.** The ability to generate and inspect the full symbolic transition matrix for small systems (say, a 4-site ring) is pedagogically valuable. Students can see exactly why detailed balance holds, where the Metropolis criterion cancels, and what happens when it is violated. The Table of transitions for the Kawasaki ring in the paper is already an excellent pedagogical illustration.
- 4. For the LLM discovery loop (if ever implemented).** If the LLM-in-the-loop architecture is implemented, *DetailedBalanceChecker* would provide qualitatively better feedback than any statistical test. The symbolic counterexample localises the error to a specific algebraic term, which the LLM can reason about. This is a strong argument for investing in the tool, but requires a working implementation to demonstrate.

11 What would make the paper significantly stronger

In rough order of impact:

- 1. Find a real bug.** Apply *DetailedBalanceChecker* to a published or widely-used algorithm and find a DB violation that numerical tests would have missed. This is the single most impactful result possible.
- 2. Engage with Neklyudov et al. [11].** The involutive MCMC framework is the natural conceptual competitor. Explain why *DetailedBalanceChecker* is needed even if one adopts the involutive design principle.
- 3. Provide a Julia or Python prototype.** Even a limited Julia port (covering Kawasaki dynamics) would dramatically reduce the Mathematica barrier and show that the approach is not language-locked.
- 4. Demonstrate on a real non-Kawasaki production algorithm.** The examples currently focus on algorithms written for the paper. Applying *DetailedBalanceChecker* to an existing published implementation (e.g., a released VMMC code) and reporting the outcome, whether PASS or FAIL, would demonstrate real-world applicability.
- 5. Quantify the ergodicity and physical fidelity metrics on the examples.** The paper describes M1 and M2 but does not report them for any algorithm. Report at least one set of numbers.

6. **Be more precise about the “formal proof” claim.** Replace “formal proof” with “algebraic proof via CAS” throughout and explain what Mathematica’s FullSimplify trusts.

12 Summary verdict

DetailedBalanceChecker is genuinely unique in its specific niche: automated symbolic DB verification of physics MC algorithms with free parameters. No published tool, software package, or paper describes anything equivalent. The gap is real.

However, the gap is narrow. The physics simulation community is small, has almost no verification culture, and works in C++/Python/Julia rather than Mathematica. The statistics/ML community, which is much larger, has entirely adequate testing tools (Geweke, SBC, MCUnit) for their use cases. Formal verification communities (Isabelle, Lean) would view *DetailedBalanceChecker*’s CAS proofs as preliminary rather than definitive.

The tool would be compelling to:

- Groups developing new cluster MC algorithms (a small but real community)
- Physics simulation software maintainers who want automated tests
- Graduate students and educators in computational statistical physics

The tool would struggle to gain adoption from:

- The large Bayesian MCMC community (SBC is sufficient and more convenient)
- Production physics simulation groups (C++/Python dependency mismatch)
- Formal verification researchers (CAS $\neq$ machine-checked proof)

The publication path is real if the paper: (1) finds a genuine bug in a real algorithm, (2) engages honestly with the limitations listed here, (3) cites Neklyudov et al. and explains the relationship to involutive MCMC, and (4) maintains its current honest tone about what the tool does and does not prove. JCTC is the right venue. The paper would be improved by tightening the use-cases section to focus on demonstrated results rather than prospective applications.

References

- [1] J. Geweke, *Getting it Right: Joint Distribution Tests of Posterior Simulators*, J. Am. Stat. Assoc. **99**, 799 (2004).
- [2] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman, *Validating Bayesian Inference Algorithms with Simulation-Based Calibration*, arXiv:1804.06788 (2018).
- [3] A. Gandy and G. Scott, *Unit Testing for MCMC and Other Monte Carlo Methods*, arXiv:2001.06465 (2020). R package `mcunit`.
- [4] M. Wallerberger and E. Gull, *Hypothesis Testing of Scientific Monte Carlo Calculations*, Phys. Rev. E **96**, 053303 (2017).
- [5] H. Tjelmeland and J. T. Kvaløy, *An MCMC Hypothesis Test to Check a Claimed Sampler*, arXiv:2505.24400 (2025).
- [6] K. A. Merz and M. R. Shirts, *Testing for Physical Validity in Molecular Simulations*, PLOS ONE (2018).
- [7] P. Narayanan, J. Carette, W. Romano, C. Shan, and R. Zinkov, *Probabilistic Inference by Program Transformation in Hakaru*, FLOPS (2016).
- [8] M. Cusumano-Towner, F. A. Saad, A. K. Lew, and V. K. Mansinghka, *Gen: a General-Purpose Probabilistic Programming System with Programmable Inference*, PLDI (2019).
- [9] A. K. Lew, M. Cusumano-Towner, B. Sherman, M. Carbin, and V. K. Mansinghka, *Trace Types and the Metatheory of Non-Parametric MCMC*, POPL (2020).

- [10] T. Gehr, S. Misailovic, and M. Vechev, *PSI: Exact Symbolic Inference for Probabilistic Programs*, CAV (2016). <https://psisolver.org/>.
- [11] K. Neklyudov, M. Welling, E. Egorov, and D. Vetrov, *Involutive MCMC: A Unifying Framework*, ICML (2020). arXiv:2006.16653.
- [12] L. Tierney, *A Note on Metropolis–Hastings Kernels for General State Spaces*, Ann. Appl. Probab. **8**, 1 (1998).
- [13] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth, *Hybrid Monte Carlo*, Phys. Lett. B **195**, 216 (1987).
- [14] M. Kwiatkowska, G. Norman, and D. Parker, *PRISM 4.0: Verification of Probabilistic Real-Time Systems*, CAV (2011).
- [15] C. Dehnert, S. Junges, J.-P. Katoen, and M. Volk, *A Storm is Coming: A Modern Probabilistic Model Checker*, CAV (2017). <https://www.stormchecker.org/>.
- [16] J. Hölzl, *Markov Chains and Markov Decision Processes in Isabelle/HOL*, J. Autom. Reason. **59**, 345 (2017).
- [17] S. Whitelam and P. L. Geissler, *Avoiding Unphysical Kinetic Traps in Monte Carlo Simulations of Strongly Attractive Particles*, J. Chem. Phys. **127**, 154101 (2007).
- [18] R. Grosse and D. Duvenaud, *Testing MCMC Code*, arXiv:1412.5218 (2014).
- [19] S. Caracciolo and A. D. Sokal, *A New Test for Random Number Generators*, arXiv:cond-mat/9806059 (1998).